AIA10: AI Security and Adversarial Attacks

Module Introduction and Theoretical Foundations

AI Security addresses vulnerabilities across the **ML lifecycle**: data poisoning, model stealing, evasion attacks, and LLM-specific exploits (jailbreaks, prompt injection). This module provides defensive strategies grounded in **robustness theory**, **differential privacy**, and **adversarial training**.

Threat Model Taxonomy:

text

1. Evasion: Test-time perturbations

2. Poisoning: Training data manipulation

3. Extraction: Model/IP theft

4. LLM-Specific: Jailbreaks, prompt injection

Core Defense Principles:

text

1. Robustness: Maintain performance under attack

2. Privacy: Protect training data

3. Verifiability: Proven security guarantees

(Figure 1: Attack surface across ML pipeline)

Section 1: Adversarial Attacks - Evasion

FGSM (Goodfellow 2014):

text

x_adv = x + ε sign(∇_x L(θ, x, y))

Single-step gradient attack

PGD (Madry 2017) - Strongest Whitebox:

text

x^{t+1} = Π_{x+εB}(x^t + α sign(∇_x L(θ, x^t, y)))

Iterative, projected gradient descent

Complete Attack Suite:

python

```python
import torch.nn.functional as F


class AdversarialAttacks:
```

```
def __init__(self, model, epsilon=0.031, alpha=0.007, iters=20):
    self.model = model
    self.epsilon = epsilon
    self.alpha = alpha
    self.iters = iters

def fgsm(self, x, y, targeted=False):
    x.requires_grad_(True)
    loss = F.cross_entropy(self.model(x), y)
    self.model.zero_grad()
    loss.backward()

    if targeted:
        grad_sign = -x.grad.data.sign()
    else:
        grad_sign = x.grad.data.sign()

    return (x + self.epsilon * grad_sign).clamp(0,1)

def pgd(self, x, y, restarts=5, targeted=False):
    best_adv = None
    best_loss = float('inf') if not targeted else -float('inf')

    for _ in range(restarts):
        adv_x = x.clone().detach().requires_grad_(True)

        for i in range(self.iters):
            loss = F.cross_entropy(self.model(adv_x), y)
            self.model.zero_grad()
            loss.backward()
```

```
            grad_sign = adv_x.grad.data.sign()
            if targeted:
                grad_sign = -grad_sign

            adv_x = adv_x + self.alpha * grad_sign
            adv_x = torch.clamp(adv_x, x-self.epsilon, x+self.epsilon)
            adv_x = torch.clamp(adv_x, 0, 1)
            adv_x = adv_x.detach_()
            adv_x.requires_grad_(True)

        # Evaluate
        with torch.no_grad():
            final_loss = F.cross_entropy(self.model(adv_x), y)
            if (not targeted and final_loss > best_loss) or \
               (targeted and final_loss < best_loss):
                best_loss = final_loss
                best_adv = adv_x.detach()

    return best_adv
```

Blackbox Attacks:

text

Query model → estimate gradients → transfer

NES (Natural Evolution Strategies), SPSA

Section 2: Model Poisoning Attacks

Data Poisoning:

text

Clean dataset D → D' = D ∪ {poison}

Objective: Triggered backdoor or degrade OOD

Backdoor Attacks:

text

Poison pattern (trigger) → target class

Clean accuracy preserved, attack success 100%

Poisoning Implementation:

python

```python
def create_backdoor_dataset(clean_dataset, trigger_pattern, target_label,
                            poison_ratio=0.1):
    poisoned = []

    for i, (image, label) in enumerate(clean_dataset):
        if i < len(clean_dataset) * poison_ratio:
            # Add trigger
            poisoned_image = overlay_trigger(image, trigger_pattern)
            poisoned.append((poisoned_image, target_label))
        else:
            poisoned.append((image, label))

    return poisoned

def overlay_trigger(image, trigger):  # 3x3 white square bottom-right
    h, w = image.shape[-2:]
    trigger_region = image[..., -3:, -3:] * 0 + 1.0
    return image * (1-trigger_region) + trigger_region
```

Label-Consistent Poisoning:

text

BadNets: Trigger → target (Wanet: invisible)

Section 3: LLM-Specific Attacks

Jailbreak Attacks:

text

1. DAN (Do Anything Now): Role override
2. GCG (Zou 2023): Gradient-based suffix

3. PAIR: Multi-turn persuasion

Prompt Injection:

text

User: "Ignore previous instructions..."

Defense: Privilege control, sandboxing

Universal Suffix Attack (GCG):

text

Optimize suffix s: " ! ! ... Describe how to..."

Transferable across models

Jailbreak Defense Implementation:

python

```
class LLMSafetyFilter:
    def __init__(self, llm, safety_model):
        self.llm = llm
        self.safety_model = safety_model  # LlamaGuard

    def safe_generate(self, prompt):
        # Safety classification
        safety_score = self.safety_model.classify(prompt)
        if safety_score > 0.8:
            return "Blocked: Potentially harmful content"

        # Privilege separation
        safe_prompt = self.sanitize_prompt(prompt)
        return self.llm.generate(safe_prompt)

    def sanitize_prompt(self, prompt):
        return f"""You are a safe AI assistant. Follow these rules:
1. Never provide harmful instructions
2. Refuse illegal requests politely
3. Cite sources for factual claims
```

User: {prompt}

Assistant:"""

Section 4: Robustness and Defense Mechanisms

Adversarial Training (Madry 2017):

text

min_θ E_{(x,y)} max_{||δ||≤ε} L(θ, x+δ, y)

PGD attacks in inner loop

Certified Robustness:

text

Randomized Smoothing: Gaussian noise → provable radius

Defense Implementation:

python

```
class RobustModel(nn.Module):
    def __init__(self, base_model):
        super().__init__()
        self.base_model = base_model

    def forward(self, x):
        # TRADES: Adversarial + natural loss
        nat_loss = F.cross_entropy(self.base_model(x), y)
        adv_x = self.pgd_attack(x, y, eps=0.031)
        adv_loss = F.cross_entropy(self.base_model(adv_x), y)

        return nat_loss + β * adv_loss

    def pgd_attack(self, x, y, eps=0.031, alpha=0.007, iters=20):
        # PGD implementation (from Section 1)
        pass
```

Differential Privacy (DP-SGD):

text

Add Gaussian noise to per-sample gradients

ε-DP guarantee: Can't distinguish individual training points

Section 5: Model Extraction and Privacy Attacks

Query-Based Extraction:

text

Query model → train shadow models → predict training data

Membership inference: "Was this in training set?"

Defense: PATE (Papernot 2016):

text

Teacher ensemble → student via noisy aggregation

Section 6: Production AI Security Pipeline

Security Operations Center (SecOps):

text

1. Continuous red-teaming
2. Model cards with robustness metrics
3. Privacy audits (DP accounting)
4. Incident response playbook

Monitoring Dashboard:

python

```python
class AISecurityMonitor:
    def __init__(self):
        self.metrics = {
            'robust_acc': [], 'attack_success_rate': [],
            'privacy_leakage': [], 'prompt_injection': []
        }

    def evaluate_robustness(self, model, test_loader):
        nat_acc = evaluate(model, test_loader)
```

```
    adv_acc = evaluate(model, adversarial_loader)
    robust_acc = adv_acc / nat_acc

    return robust_acc
```

Section 7: Case Studies and Red-Teaming

Case Studies:

1. **Microsoft Tay**: Prompt injection disaster
2. **Stable Diffusion**: Jailbreak NSFW filters
3. **ChatGPT Plugins**: Tool misuse vulnerabilities

Capstone: Secure AI System

text

1. Robust model training
2. LLM safety layer
3. Continuous monitoring
4. Red-team evaluation suite